

DRIFTAUTO

Documentation Technique

Client lourd Java — Logiciel de gestion de données
Client léger PHP HTML — Interface web Cliente et administrateur

Par Ilies IDBAIA, Seignon ZINSOU, Tiago PALHAIS et Yacine BOURENNANE

I. CLIENT LOURD

1. Stack technique

Voici les technologies utilisées dans le projet DriftAuto :

Technologie	Détail
Langage	Java 17
Interface graphique	Java Swing
Accès base de données	JDBC (Java Database Connectivity)
Base de données	MySQL
IDE	Eclipse

2. Architecture de l'application

DriftAuto suit une architecture MVC (Modèle - Vue - Contrôleur), organisée en trois packages distincts :

- modele : gestion de la connexion à la base de données et des requêtes SQL
- vue : tous les panels Swing et les fenêtres affichées à l'écran
- controleur : classes métier représentant les entités de la base, plus les utilitaires

Ce découpage permet de séparer clairement l'affichage (vue), la logique (contrôleur) et les données (modèle).

ARCHITECTURE MVC — DriftAuto

[VUE — package vue]

VueConnexion · VueGenerale · PanelPrincipal · PanelClients · PanelMoniteurs

↕ appels / événements

[CONTRÔLEUR — package controleur]

Client · Moniteur · Voiture · Lecon · Examen · Formation · Achete · Contient · Possede · Controleur · Tableau

↕ requêtes SQL via JDBC

[MODÈLE — package modele]

BDD · Modele

↕ connexion JDBC

[BASE DE DONNÉES — MySQL : auto_ecole]

3. Description des classes principales

Classe	Package	Rôle
BDD.java	modele	Gère la connexion JDBC à la base MySQL. Contient l'URL, le login et le mot de passe de connexion.
Modele.java	modele	Exécute les requêtes SQL (SELECT, INSERT, UPDATE, DELETE) et retourne les résultats à l'application.
Controleur.java	controleur	Fait le lien entre les vues et le modèle. Traite les actions de l'administrateur.
Tableau.java	controleur	Utilitaire d'affichage : alimente les JTable Swing avec les données retournées par le modèle.
Client.java	controleur	Représente un client de l'auto-école (entité métier).
Moniteur.java	controleur	Représente un moniteur. Contient aussi le champ administrateur et mdp utilisés pour la connexion.
Voiture.java	controleur	Représente une voiture du parc (immatriculation, date achat, km).
Lecon.java	controleur	Représente une leçon de conduite avec son statut (en attente, confirmé, annulé).
Examen.java	controleur	Représente un examen lié à un moniteur et un client.
Formation.java	controleur	Représente une formation proposée (Permis B, AAC, etc.).
Achete.java	controleur	Table de liaison : formation achetée par un client.
Contient.java	controleur	Table de liaison : leçon rattachée à une formation.
Possede.java	controleur	Table de liaison : examen rattaché à une formation.
VueConnexion.java	vue	Fenêtre de connexion : saisie email et mot de passe de l'administrateur.
VueGenerale.java	vue	Fenêtre principale de l'application après connexion.
PanelPrincipal.java	vue	Panel d'accueil affiché après connexion.
PanelClients.java	vue	Panel de gestion des clients : affichage, recherche, CRUD.
PanelMoniteurs.java	vue	Panel de gestion des moniteurs.
PanelVoitures.java	vue	Panel de gestion des voitures.
PanelLecons.java	vue	Panel de gestion des leçons.
PanelExamens.java	vue	Panel de gestion des examens.
PanelFormations.java	vue	Panel de gestion des formations.

4. Modèle de données

4.1 Description des tables en SQL

La base de données s'appelle auto_ecole et contient les tables suivantes :

Table	Colonnes principales	Clés
Moniteur	numero_moniteur, nom, prenom, email_moniteur, mdp_moniteur, administrateur (boolean)	PK : numero_moniteur
Client	numero_client, pseudo, nom, prenom, email_client, mdp_client	PK : numero_client
Voiture	numero_immatriculation, date_achat, nombre_km	PK : numero_immatriculation
Formation	numero_formation, nom_formation, prix_formation, total_heures	PK : numero_formation
Lecon	numero_lecon, date_heure_lecon, statut	PK : numero_lecon FK : Moniteur, Client, Voiture
Examen	numero_examen, date_heure_examen	PK : numero_examen FK : Moniteur, Client
Achete	numero_formation, numero_client	PK composite FK : Client, Formation
Contient	numero_lecon, numero_formation	PK composite FK : Lecon, Formation
Possede	numero_examen, numero_formation	PK composite FK : Examen, Formation

Note : l'authentification de l'administrateur repose sur la table Moniteur. Le champ administrateur (boolean) vaut 1 pour les comptes ayant accès au logiciel.

4.2 Modèle Logique de données (MLD)

La base de donnée est représenté par le MLD suivant :

CLIENT (numero_client, pseudo_client, nom_client, prenom_client, date_naissance_client, telephone_client, adresse_client, code_postal_client, ville_client, email_client, mdp_client, #numero_lecon)

MONITEUR (numero_moniteur, nom_moniteur, prenom_moniteur, date_naissance_moniteur, telephone_moniteur, adresse_moniteur, code_postal_moniteur, ville_moniteur, email_moniteur, date_embauche, administrateur, mdp_moniteur, #numero_lecon)

EXAMEN (numero_examen, date_heure_examen, #numero_moniteur, #numero_client)

LECON (numero_lecon, date_heure_lecon, #numero_immatriculation)

VOITURE (numero_immatriculation, date_achat, nombre_km)

FORMATION (numero_formation, nom_formation, prix_formation, total_heures)

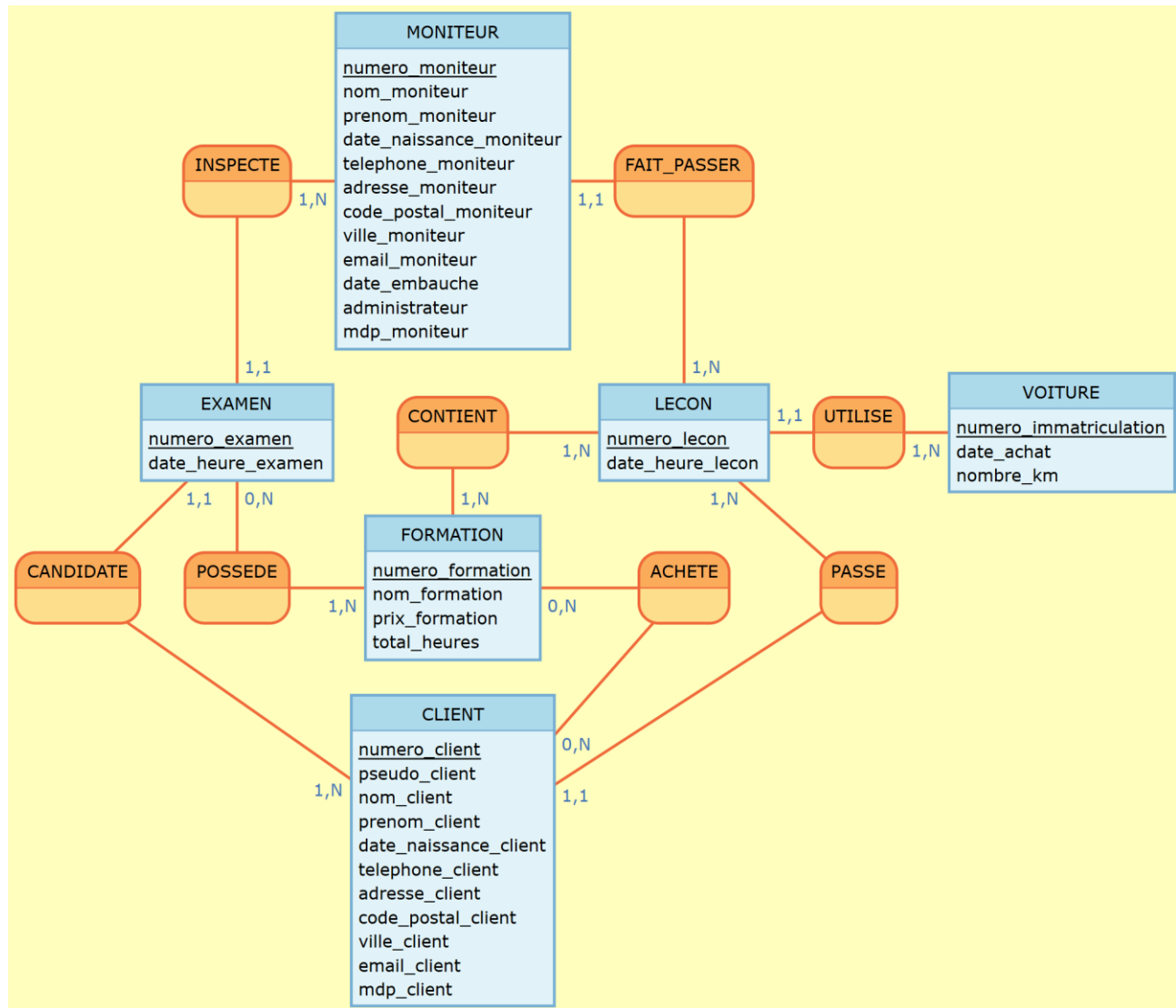
POSSEDE (#numero_examen, #numero_formation)

CONTIENT (#numero_lecon, #numero_formation)

ACHETE (#numero_formation, #numero_client)

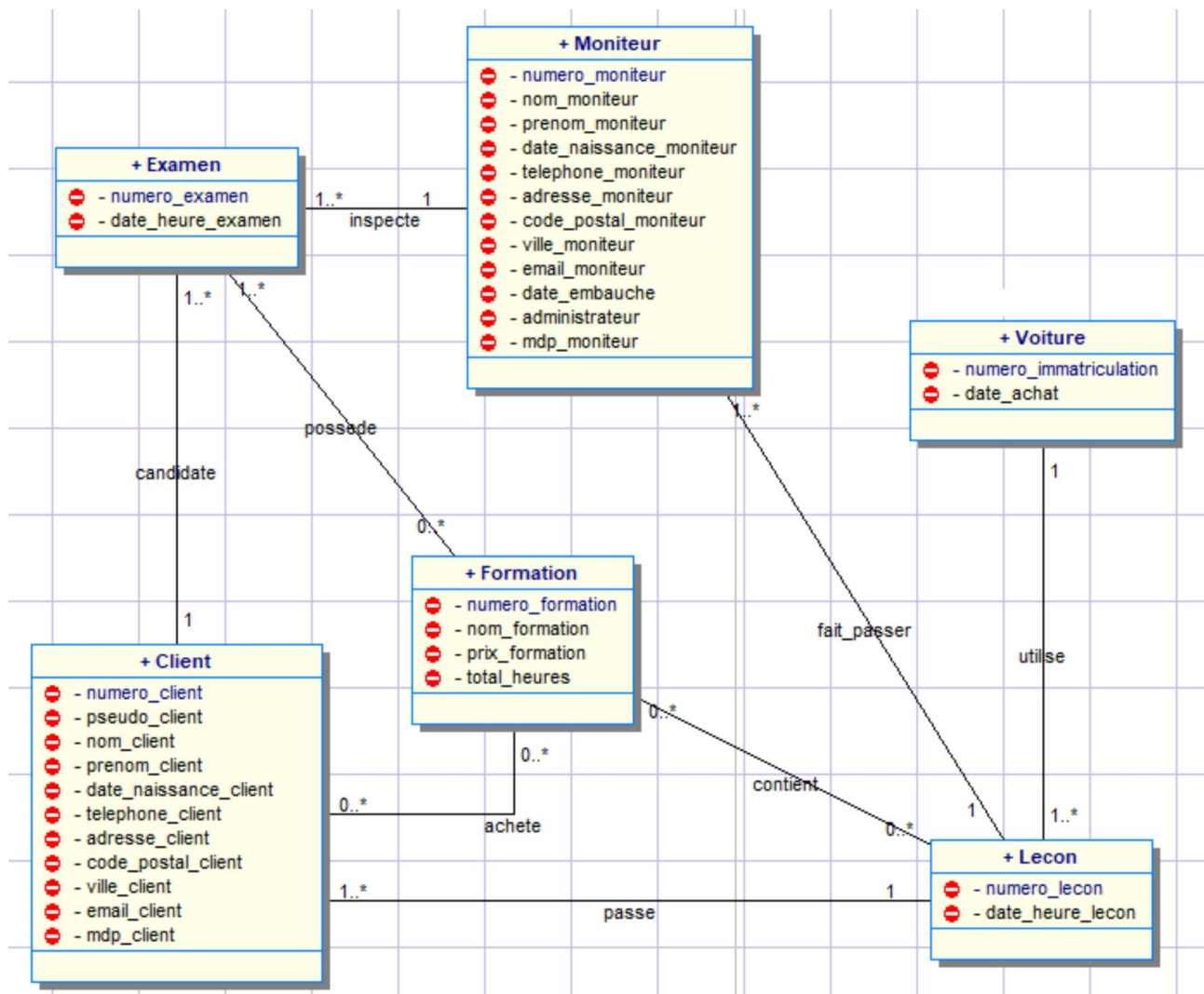
4.2 Modèle Conceptuel de données (MCD)

Voici le MCD de DrfitAuto :



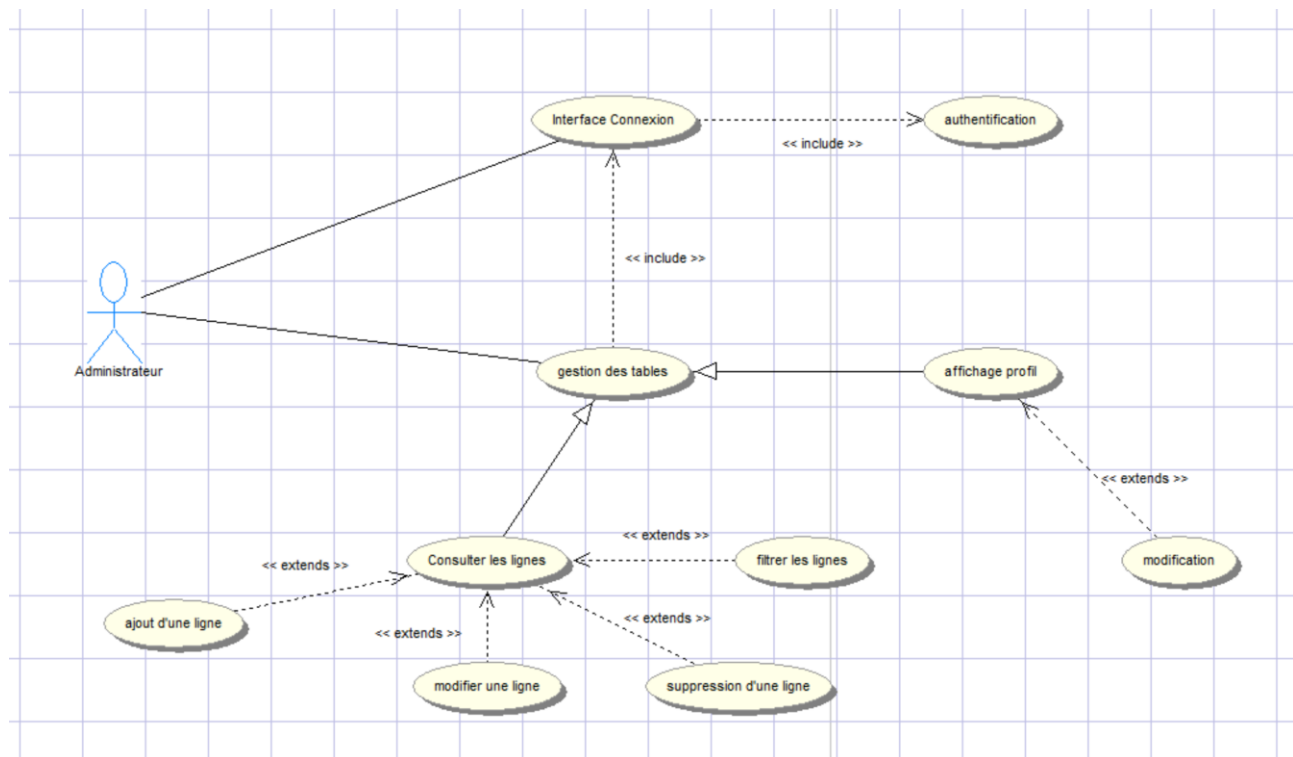
4.3 Langage de Modélisation Unifié (UML)

Voici le Diagramme de Classe caractérisant l'UML de la base de donnée :



4.4 Diagramme de Cas d'Utilisation (DCU)

Voici le Diagramme de Cas d'utilisation du client lourd :



5. Lancement

- Étape 1 : importer le projet dans Eclipse
- Étape 2 : exécuter le fichier auto_ecole.sql dans MySQL pour créer la base
- Étape 3 : lancer AutoEcole.java comme application Java
- Étape 4 : se connecter avec un compte Moniteur ayant administrateur = 1

Compte de test disponible : jean.dupond@driftauto.mo / mdp123

II. CLIENT LEGER

1. Stack technique

Voici les technologies utilisées dans le projet DriftAuto :

Technologie	Détail
Langage	PHP, HTML, CSS,
Base de données	MySQL
IDE	SublimeText

2. Architecture de l'application

ARCHITECTURE MVC — DriftAuto

[VUE — dossier vue]

[footer.php](#) · [header.php](#) · [vue_connexion.php](#) · [vue_insert_utilisateur.php](#) · (dossier) [vues_voiture](#) · [vues_client](#) · [vues_moniteur](#) · [vues_lecon](#)

↕ appels / événements

[CONTRÔLEUR — dossier controleur]

[Gestion_client.php](#) · [gestion_moniteur.php](#) · [gestion_voiture.php](#) · [gestion_lecon.php](#) · [erreur.php](#) · [controleur.class.php](#)

↕ requêtes SQL via PDO

[MODÈLE — dossier modele]

Modele.class.php

↕ connexion via PDO

[BASE DE DONNÉES — MySQL : auto_ecole]

3. Modèle de données – Diagramme des cas d'utilisations

Voici le Diagramme de Cas d'utilisation du client léger :

